Nutrition & Recipe Analytics API Interface Technical Documentation

1. Description of document purpose and operating environment

This API document aims to explain in detail the calling specification, data interaction format and error handling semantics of the Nutrition & Recipe Analytics system interface. The whole system follows RESTful architecture style, and the default input and output formats of all interfaces are JSON. In the local development and test environment, developers can start the service by executing the `uvicorn app.main:app --reload` command, and the default access portal is http://127.0.0.1:8000. The system connects to the SQLite database locally by default, and can switch to PostgreSQL seamlessly by configuring the `DATABASE_URL` environment variable during production deployment. Developers can also access the `/docs` path to enter the interactive Swagger UI panel for real-time interface debugging.

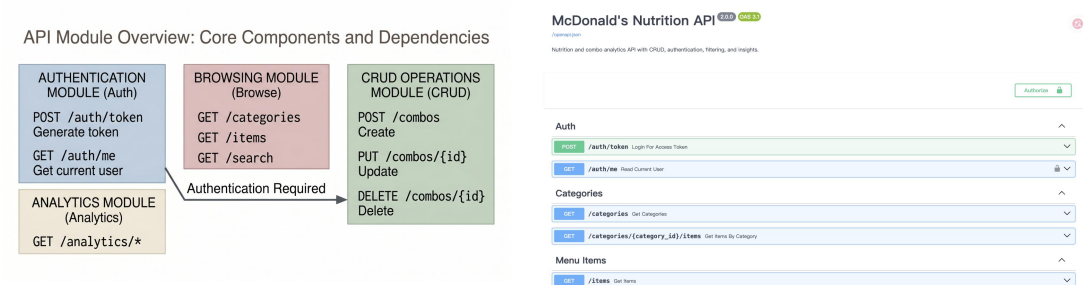

Figure A1: API Module Overview (left) and Swagger UI Home Page (right), illustrating the architectural relationships across the Auth, Browse, CRUD, and Analytics modules

2. Identity authentication mechanism and security access control

In order to ensure the security of core system data (such as customized packages), this system introduces an authentication mechanism based on JSON Web Token (JWT), which realizes the security boundary of "free inquiry of public data and controlled writing of core data".

The developer needs to initiate a request to the authentication endpoint `POST /auth/token` to obtain access credentials. This interface requires the client to submit the legal `username` and `password` in the form of `application/x-www-form-urlencoded`. After verification, the server will issue JSON data containing the access token, as shown below:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5c...",
  "token_type": "bearer"
}
```

After obtaining the token, when the client requests any protected write interface (such as creating or deleting a package), it must append the `Authorization` field to the HTTP request header and set its value to `Bearer <access_token >`. If the token is missing or invalid, the system will deny service and return to `401 Unauthorized`.

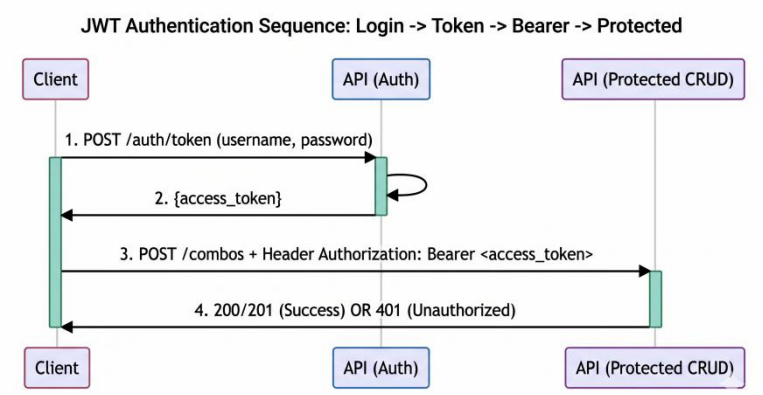

Figure A2: JWT Authentication Sequence Diagram, illustrating the complete request lifecycle from Login and Token Issuance to Bearer Request and Protected Endpoint access

3. Interaction between core data model and business logic

The bottom layer of the system is built based on four core relational entities: `Category`, `MenuItem`, `Combo` and `ComboItem` as an intermediate mapping. This design can not only clearly express the ownership relationship of the basic ingredients, but also accurately record the complex many-to-many combinations in the user-defined package through the `quantity` field in the `ComboItem` table. When interacting with API, the system will check the legitimacy of association between these entities in real time, and dynamically aggregate and calculate the total calories and total proteins when reading the package information to ensure the final consistency of the data.

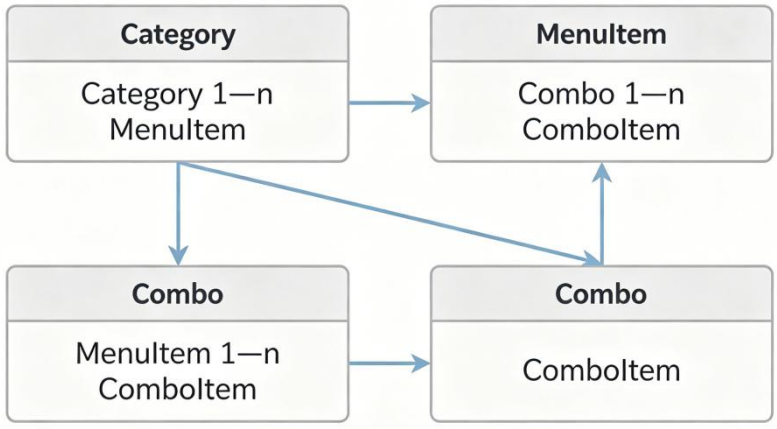

Figure A3: Entity Relationship (ER) Diagram, detailing the cardinality constraints between Category (1) and MenuItem (N), as well as the relationships bridging Combo (1), ComboItem (N), and MenuItem (1)

4. Core Endpoint and Interaction Example Description

The system provides abundant resource operation and data retrieval endpoints. The following lists in detail the core interfaces and their calling specifications that meet the complete CRUD life cycle and advanced analysis functions.

4.1 retrieval of dishes and screening of nutrients (GET `/items/search`)

The open interface allows users to filter the food library according to specific nutritional indicators. The client can narrow the search scope by appending Query Parameters to the URL, for example, passing the integer max_calories parameter to limit the maximum calories of dishes, or passing the min_protein parameter to screen the ingredients that meet the standards in protein. When the request is successfully processed, the system will return a 200 OK status code with a JSON array with the following structure:

```
[
  {
    "item_id": 101,
    "name": "Grilled Chicken Breast",
    "calories": 165,
    "protein": 31,
    "category_id": 2
  }
]
```

4.2 Custom Package Creation Operation (POST /combos)

As the core protected write endpoint of the system, the interface requires that the request header must contain a valid JWT token. The client needs to submit structured JSON data in the request body, which clearly contains the package name and description description of string type, and an array named items. Each object in the array must indicate the item_id of the effective ingredients and its corresponding quantity quantity.

Example of Request Payload (request payload):

```
{
  "name": "High Protein Muscle Builder",
  "description": "Optimized meal tailored for post-workout recovery.",
  "items": [
    { "item_id": 101, "quantity": 2 },
    { "item_id": 105, "quantity": 1 }
  ]
}
```

After checking the existence of ingredients and no name conflict, the system will persist the data and return the 201 Created status code. The returned JSON response not only contains the generated unique identifier combo_id, but also outputs the total_calories and total_protein dynamically calculated by the server, as shown below:

```
{
  "combo_id": 50,
  "name": "High Protein Muscle Builder",
  "total_calories": 450,
  "total_protein": 65,
  "created_at": "2026-04-20T14:30:00Z"
}
```

4.3 User-defined package deletion operation (delete/combo/{combo _ id})

When the package data needs to be discarded, the authorized user can call this endpoint. The request URL must contain the unique integer identifier of the target package (that is, the path parameter combo_id). If the target resource exists and the permissions are verified, the system will clear the corresponding records and all intermediate table mappings in the database. After the operation is successful, the server will return a status code of 204 No Content, indicating that the request has been executed and there is no need to return the subject data.

4.4 comprehensive analysis panel of nutritional efficiency (get/analytics/combo-scorecard)

This endpoint shows the scalability of the system in data insight. This interface does not need additional parameters, it will traverse all the created packages in the background, and introduce a derivative index named protein_density. This index is obtained by dividing the total protein mass by the total calories, which is used to objectively evaluate the protein conversion efficiency in unit calories. After the server calculates, all packages will be arranged in descending order according to this indicator, and the ranking report in the following format will be returned to the client:

```
[
  {
    "combo_id": 50,
    "name": "High Protein Muscle Builder",
    "protein_density": 0.144,
    "rank": 1
  },
  {
    "combo_id": 12,
    "name": "Standard Vegan Salad",
    "protein_density": 0.085,
    "rank": 2
  }
]
```

Figure A4: Nutritional Efficiency Analytics Output, featuring a sample data visualization report of the protein density rankings

5. Parameter verification, error handling semantics and quality verification

At the entrance level, the system relies on Pydantic library to implement strict data verification rules. In order to ensure the efficiency of front-end debugging, all exceptions are intercepted and converted into JSON error responses with unified structure. In the use of standard HTTP status codes, the system specifications are as follows: 400 Bad Request stands for business rule conflict; 401 Unauthorized prompts authentication failure; 403 Forbidden means unauthorized access; 404 Not Found means that the path parameter points to a non-existent resource (such as invalid combo _ ID); Internal data exception throws 500 Internal Server Error.

Especially, when the JSON data format or field type submitted by the client does not meet the interface definition, the system will return a 422 Unprocessable Entity error. The advantage of this kind of error response is that it provides accurate context positioning. As shown in the following example, the system clearly points out the specific parameter level that causes the error through the path field, which greatly reduces the debugging cost:

```
{
  "error": "Validation Error",
  "detail": "quantity must be strictly greater than 0",
  "path": "body -> items -> 0 -> quantity"
}
```

In order to verify the reliability of the above behavior from the engineering point of view, this system uses Pytest framework to build a comprehensive automated test case library, covering authentication bypass test, core resource CRUD full link verification and boundary parameter input test. Developers can check the test pass rate index by executing pytest -q in the root directory.

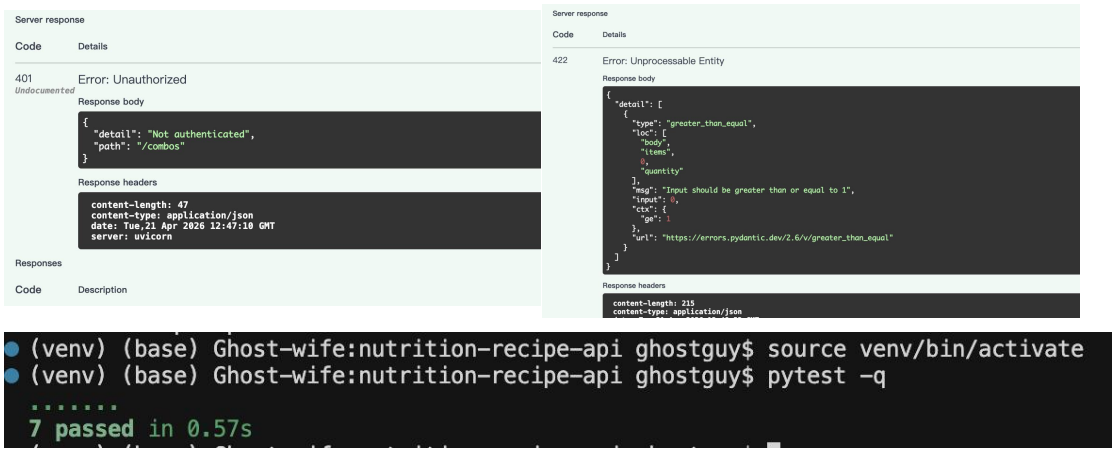

Figure A5: Common Error Response Comparisons and Automated Testing Output, displaying side by side 401 and 422 error payloads alongside successful Pytest console execution results